

What Is the Smallest (AI) Platform That Could Possibly Work?

The most common AI request I see is access to an LLM. Giving an engineer an API key takes minutes and solves exactly one problem. What the key does **not** do:

show finance which team
created which cost

help compliance understand
where the data goes

give incident response a
system it can observe

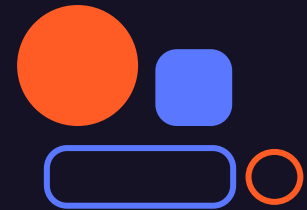
tell anyone whether the agent
still works

That work arrives later — sometimes weeks after the part that felt like the job. It lands on us, and it lands again with the next team.

Two questions for today

- Does that repeated work justify a platform used by several teams?
- If it does, what is the smallest platform that could possibly work?

This is an introduction to how far I got. I am looking for reactions and disagreement, not a decision.



No single request is a platform problem

A collection of experiments does not become a platform problem just because the experiments use AI.

It becomes a platform problem when several teams repeatedly need the **same** access controls, cost attribution, evaluation, observability, or audit evidence — and solving each case separately creates more work, inconsistent controls, or unnecessary risk.

The test is not how much AI activity we have

Does the same problem keep appearing?

What happens if we ignore it?

Would solving it once for several teams remove more work than it creates?

Even when the answer is yes, the answer may be to extend a platform we already own. A dedicated AI platform is justified only when our existing systems cannot solve the repeated problem without every team rebuilding the same AI-specific parts.

What is actually repeating for us

Deployment support repeats

Analysts and product managers build something with a coding agent — often a dashboard. It works on their machine. Making it available to others means the company runtime and the service contract in the developer portal, which still requires engineering experience.

So we help. Manually. Again.

Signal: the same setup and translation work returns with every new builder.

Access without controls repeats

Engineers build AI-powered Slack apps and ask for an LLM key. The request itself is cheap. The key answers none of the operational questions from slide 1.

Signal: easy access creates the same unanswered questions every time.

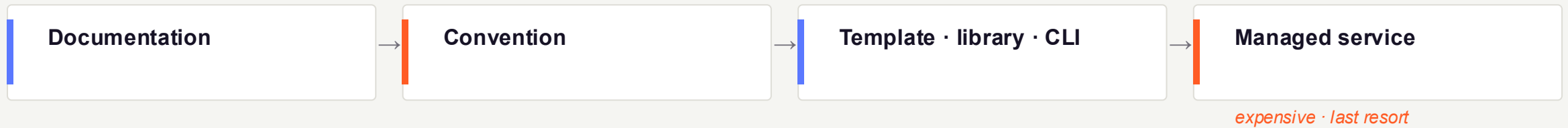
Where we already stand

The AI Playground removes part of the first pattern: a person names an agent in the developer portal and receives a repository with a working agent deployed to test, plus a starter prompt for a coding agent. What it removes is the repeated setup, the allowed technologies, and the practices we want people to inherit as skills.

Pattern one was real, and a template was the right size of answer for it. Pattern two is still open — and it is the one producing the questions from finance, compliance, and incident response.

Build less than you think you need

The order matters. Documentation → convention → template, library, or CLI → managed service. Build the service only when the simpler interventions cannot remove the repeated work.



Teams also repeat work because they are still learning. Centralizing too early turns experiments into a service nobody needs. Before building a shared service, ask whether the repeated work can be **removed** rather than served — the apparent platform problem may be unclear ownership, missing training, or a workflow that should not exist.

I have paid for getting this wrong — though not with a platform

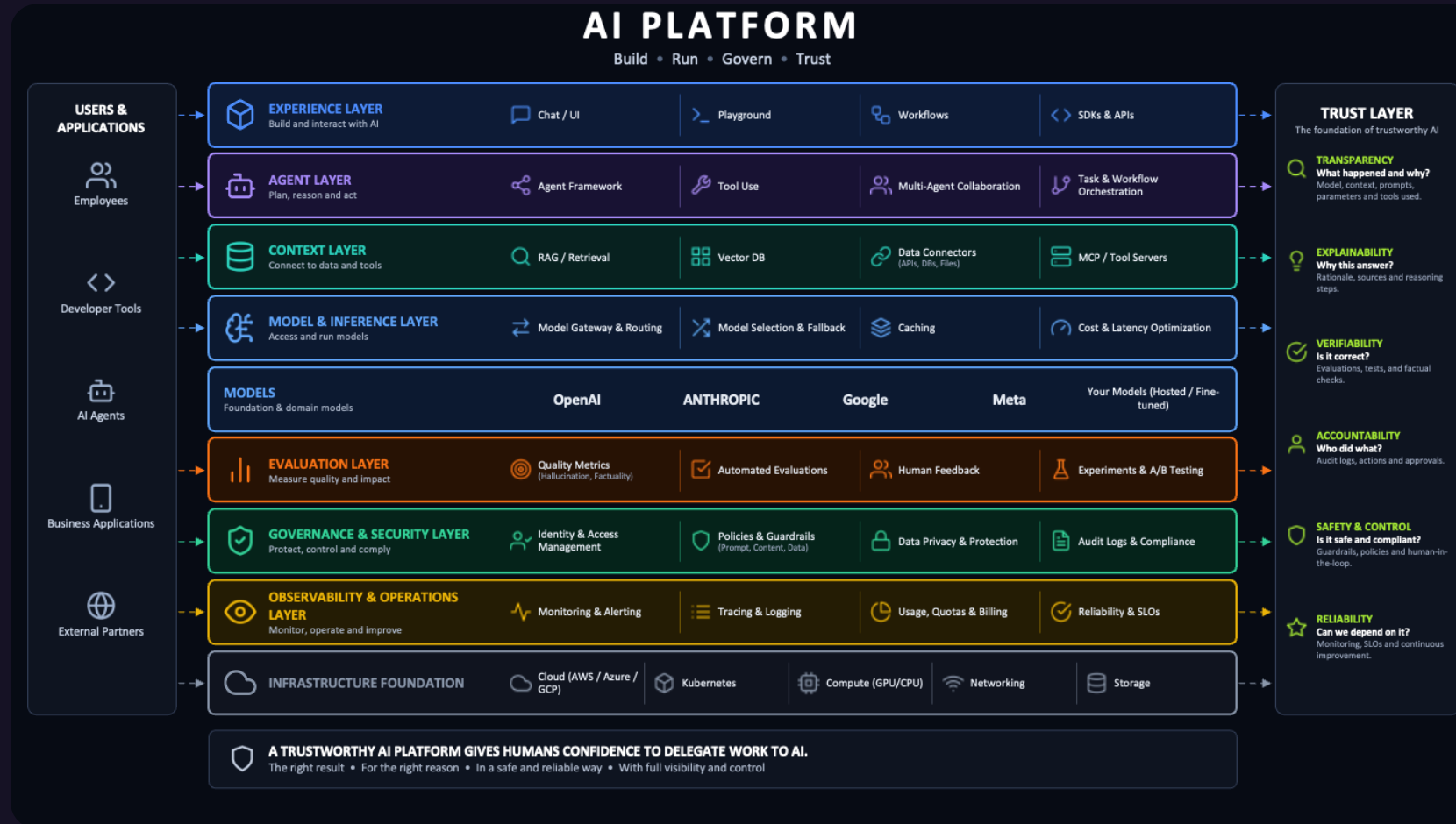
Manifold was an AI coding IDE for individual developers. I spent six months of evenings on it, convinced someone needed it. Nobody did. The software was not the problem. The problem was that I treated my own conviction as evidence that the demand existed.

An internal platform fails the same way, but more quietly. A team that stops using a platform rarely says so — it routes around the platform and keeps working. We keep the mandate and lose the users, so the mandate stops being evidence that anyone needs the thing.

That is why I no longer trust the sentence "teams need this" when I am the one saying it. Everything after this slide argues for building less than I think is necessary.

What a complete AI platform looks like

The picture most of us have in mind when someone says "AI platform" — Build · Run · Govern · Trust



Nothing here is wrong

Every box is a real capability that someone has to provide. That is what makes it dangerous as a plan: about forty capabilities, and it reads as a roadmap.

So the question is not which layers exist

- Which do we already have, under a different name?
- Which belong to the team building the agent?
- Which are not needed yet?
- What is the cheapest thing that makes the trust column answerable per agent?

Nine layers, from experience down to infrastructure — with the trust layer running down the right side.

What "smallest" has to mean

Counting features, services, or lines of configuration makes a config file look smaller than a gateway. Convenient, but not the measure that matters.

The smallest useful AI platform is the system that creates the least total work, lets a defined group complete one valuable workflow, meets the minimum required controls, and remains cheaper to change or remove than the repeated work it removes.

Outcome

Can the intended user finish the job?

Risk

Are the minimum controls there for the possible harm?

Total work

Is central and local work reduced?

Change and removal

Can it be retired at an acceptable cost?

"Smallest" is the simplest thing that passes all four — for one particular workflow.



A versioned agent contract

The smallest platform I can justify is a narrow service built around a **versioned agent contract**: a configuration file stored with the code that describes the agent and the systems it uses. The contract alone is not the platform. The contract **and** the service are: CI validation, connections to the systems that own operational facts, and a view in the developer portal.

Scope for version 0.1: one simple, internal, read-only agent. Not a general contract for every kind of agent.

Prototype 0.1 · agent.yaml

```
agent:
  name: settlement-explainer
  owner: payments
  purpose: explain settlement deviations
  lifecycle: experimental
  risk: internal-read-only
  allowed_data: internal
  model_access: approved-eu-provider
  runtime: application-platform
  cost_id: payments-settlement
  trace_id: settlement-explainer
  evaluation: evals/settlement.yaml
  incident_contact: payments-on-call
  disable: runbooks/disable-agent.md
```

Every field exists to make one internal, read-only workflow accountable and observable. If a field cannot be justified for **this** workflow, it does not go in yet.

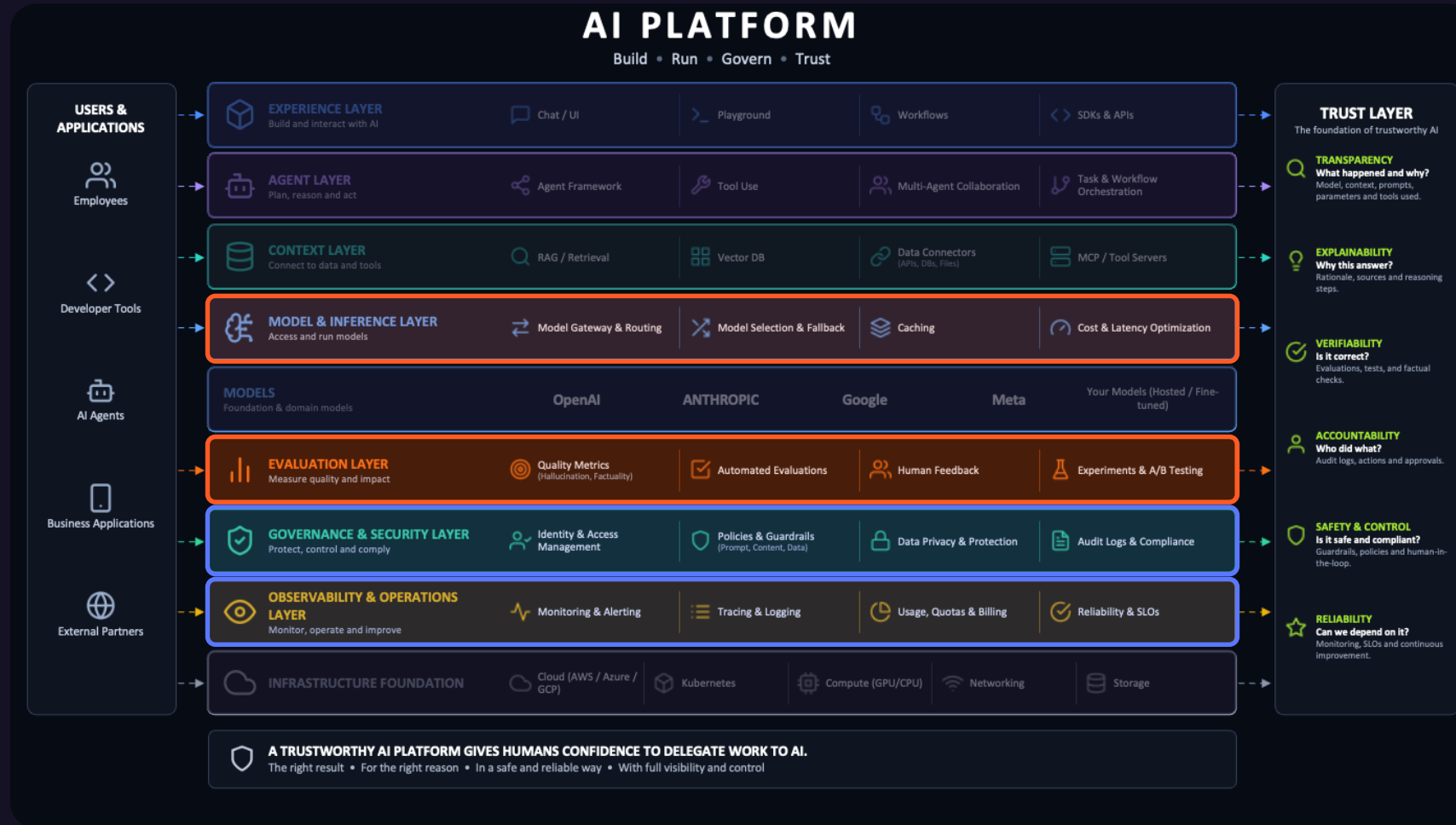
How it works

- 01 An engineer creates or changes the agent and its contract in the same repository
- 02 CI validates identity, ownership, risk, data class, operational references, incident contact and disable instructions
- 03 Delivery, evaluation, observability and billing systems continue to produce and own their operational facts
- 04 The developer portal brings declarations, current facts and links to their sources into one view
- 05 The agent keeps running on the existing application runtime

No new runtime, no AI gateway, no evaluation service, no new GUI to start.

The same picture, marked up

One question per layer: who owns this, and do we need it yet?



WHO OWNS IT

- Contract declares it; existing systems own the facts
- Declared reference; the rest waits for a signal

Already have it, the team owns it, or untouched

That is the whole boundary: one file, CI validation, and a portal view — the cheapest thing on this diagram to walk away from. The other eight layers are already owned, belong to the team, or are waiting.

Transparency, accountability, verifiability and safety now trace back to a declared field plus the system that owns the fact.

The contract connects decisions. It does not enforce them.

Responsibility	Owner
Decide which outcome is required	The person accountable for the harm or obligation
Define representative cases and evidence	The team that understands the workflow
Encode and validate the rule	The platform team
Prevent a disallowed action	A system with the context and authority to stop it

A gateway can enforce allowed models. A deployment system can require evaluation evidence before release. A downstream service must authorize the business action.

A YAML file cannot stop an agent from using a credential with too much access.

Workflow-specific decisions stay with the team: model choice, prompts, business rules, authorization, acceptance levels, alerts, and when to release or roll back. When teams still need the same manual help, the declared information becomes unreliable, or every system needs a custom adapter — the next addition (generator, gateway, authorization service, evaluation service, runtime) is a response to that signal, not a roadmap item.

The condition on anything we add after the contract: it states its exit cost before we build it. The same standard applies to this platform — reduce, replace or remove it when something simpler achieves the same result.

What I don't know yet

Prototype 0.1 has one problem worth stating directly: **no team has used it**. It is a file I wrote to find out whether the idea holds together — which is the same kind of evidence I said on slide 4 that I no longer trust.

So the open question is not what the contract should contain. It is how a contract nobody asked for reaches its first team. I see three ways and I do not know which is right.

Put it in the template

What it buys Fastest adoption. Every agent generated through the internal template gets a contract by default; teams receive it without asking

What it costs Nobody chose it. The adoption numbers would look good and mean nothing — the exact failure mode from slide 4. And if CI validates it and audit depends on it, no team can ship without one

Wait for the pull

What it buys Real demand. We build when a team asks for something it enables: cost attribution, audit evidence, or a way to disable an agent during an incident

What it costs The request arrives after agents are already in production — the most expensive moment to introduce a contract, and the least likely to get engineering time

Don't build it

What it buys Cheapest. Add the fields to the service contract the developer portal already uses. No new system to own, nothing to leave

What it costs Puts AI-specific facts into a system that was not designed for them. Unclear whether that is reuse or a problem that grows slowly

The first option is the one I want to pick, and wanting it is the reason I have not.

What would help me most

Which of the three would you pick — and where does it break in your context? A concrete objection beats agreement.

What would change my mind

A team that was handed a contract by default and actually used it. If you have been here before, what did it cost?